

GEMINI

Helix Constitution — Universal GEMINI.md

Field	Value
Revision	1
Created	2026-06-09
Last modified	2026-06-09T00:00:00Z
Status	active
Status summary	New Gemini CLI carrier of the Helix Constitution — added by §11.4.140 (universal action-prefix system) so Gemini CLI gets the LAYER-1 action-prefix recognition floor out of the box, plus the §11.4.141 token-efficiency mandate mirror. This file is one of the four canonical agent context carriers (CLAUDE.md, AGENTS.md, QWEN.md, GEMINI.md) the constitution maintains per §11.4.35; the textual identity of the action-prefix recognition block across all four IS the LAYER-1 universality. This GEMINI.md is a thin carrier — the full universal rule set is in Constitution.md (canonical) and the sibling carriers; Gemini CLI reads this file as its context carrier.
Issues	none
Issues summary	—
Fixed	§11.4.140 + §11.4.141 mirrors
Fixed summary	Carrier created in lockstep with the Constitution.md §11.4.140 + §11.4.141 additions.
	—

Field	Value
Con- tinu- ation	

How inheritance works

This GEMINI.md is the **Gemini CLI carrier** of the Helix Constitution submodule. It is one of the four canonical agent context carriers (constitution/CLAUDE.md, constitution/AGENTS.md, constitution/QWEN.md, constitution/GEMINI.md) — these ARE the canonical root per §11.4.35, NOT consumer extensions. The full universal rule set lives in constitution/Constitution.md; the sibling carriers (CLAUDE.md / AGENTS.md / QWEN.md) carry the same content for their respective agents. A consuming project's own repo-root GEMINI.md (if any) opens with the inheritance pointer and adds only project-specific rules; THIS file is the canonical carrier and does not.

For agents driven from a consuming project, the project's repo-root carrier SHOULD start with a clearly-marked inheritance pointer:

```
## INHERITED FROM constitution/GEMINI.md
```

All rules in ``constitution/GEMINI.md`` (and the ``constitution/Constitution.md`` it references) apply unconditionally. The project-specific rules below extend them.

Action-prefix recognition (§11.4.140 — LAYER-1)

The block below is the **canonical LAYER-1 action-prefix recognition instruction**, embedded VERBATIM (identical text) into every agent context carrier the constitution maintains. That textual identity across all four carriers IS the LAYER-1 universality: Gemini CLI reads this block via this carrier and applies the prefix expansion itself, so the action-prefix system works on Gemini CLI with zero extra setup.

§11.4.140 — Universal action-prefix system (ACTION_NAME ::) (User mandate, 2026-06-09; GRAMMAR_ADDENDUM 2026-06-09). When a user prompt's FIRST non-blank line starts with a recognised action prefix, you MUST: (1) look the action token up in the action registry constitution/actions/registry.yaml (or

\$HELIX_ACTION_REGISTRY); (2) if it is a registered action, REPLACE the prefix with that action's expansion text and apply its rules; (3) execute the remainder of the prompt under the expanded instruction. **Four EQUIVALENT forms** — same action, same expansion, same execution: (1) ACTION_NAME :: <rest> (bare ::), (2) PREFIX::ACTION_NAME :: <rest> (namespaced ::), (3) /ACTION_NAME <rest> (bare slash), (4) / PREFIX::ACTION_NAME <rest> (namespaced slash). Thus BACKGROUND :: x ≡ DEFAULT::BACKGROUND :: x ≡ /BACKGROUND x ≡ /DEFAULT::BACKGROUND x. PREFIX is an action NAMESPACE; the reserved default namespace is **DEFAULT**, and an action runs WITH or WITHOUT the prefix. Grammar (all hold): anchored at the FIRST non-blank line only (mid-prose tokens never match); the action token AND the namespace are UPPERCASE-only [A-Z][A-Z0-9_]* (lowercase never matches); the namespace separator :: inside the token carries NO surrounding spaces (PREFIX::ACTION_NAME), DISTINCT from the action-body separator " :: " (one ASCII space on each side of :: — avoids C++ Foo::Bar, YAML key: value, URLs) in forms 1/2 and the slash-body separator (one space) in forms 3/4; stacked prefixes (A :: B :: rest) apply outer-to-inner, left-to-right (expand A, re-scan, expand B, then the residual is the task); a leading \ escapes the prefix for BOTH the :: and the slash form (\BACKGROUND :: x, \ /BACKGROUND x — treat literally, strip the backslash, NO expansion) so action names can be discussed. **Conflict rule (slash form):** / ACTION_NAME (form 3) is honored as the action ONLY when ACTION_NAME (case-folded) does not collide with a built-in/host slash command (registry slash_bare: auto + slash_conflicts: [...]); form 4 (/PREFIX::ACTION_NAME) is ALWAYS unambiguous and always honored. An unknown token that matches the grammar shape (any of the 4 forms) but is NOT registered is NEVER silently expanded or silently dropped — ask which registered action was meant (§11.4.66 / §11.4.105) or treat it as a literal prompt, NEVER invent an expansion (§11.4.6); any prompt not satisfying the grammar is an ordinary prompt and the system is a no-op. The registered action **BACKGROUND** expands to: *"The following prompt that we will provide MUST BE executed in background in parallel with all main work streams using the sub-agents-driven development approach! All work done MUST PRODUCE rock solid evidence covered with hard physical proof(s) that all done is working as expected and as specified without any false results and without any bluff!"* (composes §11.4.20 / §11.4.70 subagent-driven, §11.4.58 / §11.4.103 parallel streams, §11.4.89 background execution, §11.4.5 / §11.4.69 / §11.4.107 captured physical evidence, §11.4 anti-bluff). The system is UNIVERSAL (every CLI agent reads this block via its context carrier per §11.4.35), extensible (new action = new registry row), decoupled + reusable (§11.4.28), and loads out-of-the-box. Classification: universal (§11.4.17). **Canonical authority:** constitution submodule

Constitution.md §11.4.140. Non-compliance is a release blocker. No escape hatch — no --skip-action-prefix, --ignore-prefix, --no-registry, --invent-expansion-OK, --single-layer-only flag.

Token-efficiency mandate (§11.4.141)

§11.4.141 — Token-efficiency mandate (research-derived + operator mandate, 2026-06-09). Every project worked on by AI coding agents **MUST** cut token spend (input AND output) toward **30-40% of current (a 60-70% reduction)** WITHOUT degrading quality/performance/safety or breaking any existing mechanism, via a composable, safety-ranked measure set: (1) **prompt-cache the static governance prefix** — the always-loaded governance forms a byte-stable cache-breakpointed prefix with no volatile bytes ahead of it; cache reads cost $\sim 0.1 \times$ base input (the dominant cost driver — measured $\sim 170K$ tokens of governance re-sent every turn, externally corroborated by Claude Code issue #24147); caching is transparent so it removes no rule, weakens no gate, changes no verdict — only billing (PRIMARY, biggest + safest lever); the operative discipline is to **batch governance edits and keep CLAUDE.md + the constitution byte-stable mid-session** — every governance edit busts the prompt cache (BASELINE.md finding); (2) **subagent model-tiering + output-to-file** — mechanical non-judgment work (search/grep/status/doc-export/read-only probes) to a Haiku-class model, the strong model RESERVED for all reasoning/verdicts/fix-design (§11.4.102)/code-review (§11.4.125)/demotion (§11.4.7), large output persisted to a file not an inline 350-520K-token transcript; the cheap model never emits a PASS so §11.4.50 + anti-bluff are untouched; (3) **thin always-loaded INDEX + on-demand detail** — concise index (one line per fix/anchor, EACH carrying the literal 11.4.N token so propagation gates pass) with the canonical full text kept gate-scanned in constitution/Constitution.md and reachable in one hop — a de-duplication realising §11.4.35, never a deletion; (4) **Code-Graph/retrieval-first over full-file loading** (§11.4.78/§11.4.79); (5) **output-token reduction** — terse status + effort: "low" on the mechanical allowlist only; (6) **tool-call batching + no re-reads**; (7) **compaction/context-editing for long sessions. Mandatory measured proof:** a token-accounting harness measures tokens-per-development-cycle BEFORE vs AFTER on a frozen deterministic workload from the authoritative usage object (input/cache_read/cache_creation/output split; NEVER tiktoken, NEVER the client-side cost estimate), reproduced N times (§11.4.50), pass = $AFTER \leq 40\%$ of BEFORE OR the measured best-safe reduction with a cited cold-cache reason; the AFTER run **MUST** show ZERO regression on the pre-build sweep + meta-test mutation sweep + propagation gates + a strong-model reasoning probe + a cache-warm proof (`cache_read_input_tokens > 0`) — cost reduction with quality regression is a §11.4 FAIL. The headline number is the

measured reduction, never the design estimate (§11.4.6/§11.4.123). No measure may break/degrade any existing mechanism, and the rule is structured so none can. Composes §11.4.5/.6/.20/.40/.50/.58/.69/.70/.78/.79/.80/.103/.106/.123/.125/.128/§12.6/§1.1. Classification: universal (§11.4.17). Propagation gate CM-COVENANT-114-141-PROPAGATION (literal 11.4.141) + recommended gate CM-TOKEN-EFFICIENCY + paired §1.1 mutation (inject a pre-breakpoint volatile token → cache collapses → measured reduction falls below bar → gate FAILs). **Canonical authority:** constitution submodule Constitution.md §11.4.141. Non-compliance is a release blocker. No escape hatch — no --skip-token-efficiency, --no-cache-governance, --assert-reduction-without-measuring, --tier-down-reasoning, --inline-all-governance, --tiktoken-estimate-OK flag.

When in doubt

- Read constitution/Constitution.md for the canonical text of every rule.
- Cross-reference the sibling carriers (CLAUDE.md / AGENTS.md / QWEN.md) — they carry the same universal content for their respective agents.
- If still unclear, ask the operator. Do NOT guess. Do NOT bluff (§11.4.6).